Task 1a: Test results

```
/Users/alexr/PycharmProjects/SoftwareTech2/.venv/bin/python /
Testing with 10000 elements, search and delete value: 657029
Python list insert at beginning: 0.000004 s
Python list search: 0.000023 s
Python list delete: 0.000041 s
Singly Linked List insert at beginning: 0.000001 s
Singly Linked List search: 0.000066 s
Singly Linked List delete: 0.000140 s
Doubly Linked List insert at beginning: 0.000001 s
Doubly Linked List search: 0.000062 s
Doubly Linked List delete: 0.000109 s
```

```
/Users/alexr/PycharmProjects/SoftwareTech2/.venv/bin/python /
Testing with 50000 elements, search and delete value: 874748
Python list insert at beginning: 0.000028 s
Python list search: 0.000116 s
Python list delete: 0.000189 s
Singly Linked List insert at beginning: 0.000002 s
Singly Linked List search: 0.000303 s
Singly Linked List delete: 0.000633 s
Doubly Linked List insert at beginning: 0.000002 s
Doubly Linked List search: 0.000304 s
Doubly Linked List delete: 0.000498 s
```

As can be seen by the screenshots provided, both single and double linked lists are faster for insertion at the beginning than python lists (as is logical, since O(1) is less complex than O(n); we do not need to change anything about other elements in a linked list except for the current first element, whereas we need to change every element's index in a python list). However, a python list is quicker for searching and deleting elements. This is due to the need to collect the .next variable when looping through the linked list, which is unnecessary in the built-in python

list. A double linked list is also faster for deletion than single linked lists, as we have to assign current and previous nodes in a single linked list, changing them after each iteration, whereas a node in a double linked list already has the .prev attribute, meaning only a current node is changed in each iteration.

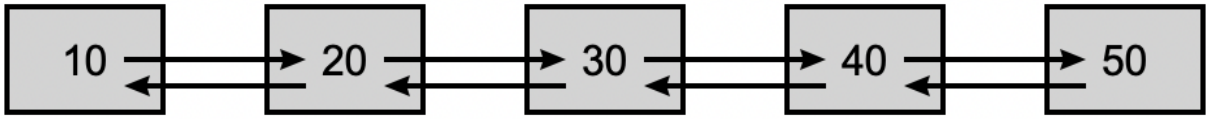
Task 1b: Test results

```
/Users/alexr/PycharmProjects/SoftwareTech2/.venv/bin/python /
Benchmarking with 10000 values
Stack (list) Push: 0.000103 s
Stack (list) Peek: 0.000000 s
Stack (list) Pop: 0.000226 s
Queue (deque) Enqueue: 0.000132 s
Queue (deque) Peek: 0.000001 s
Queue (deque) Dequeue: 0.000237 s
Priority Queue (heapq) Push: 0.000419 s
Priority Queue (heapq) Peek: 0.000000 s
Priority Queue (heapq) Pop: 0.002163 s
```

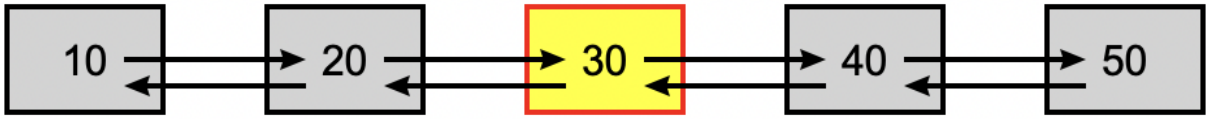
```
/Users/alexr/PycharmProjects/SoftwareTech2/.venv/bin/python /
Benchmarking with 100000 values
Stack (list) Push: 0.000804 s
Stack (list) Peek: 0.000000 s
Stack (list) Pop: 0.001755 s
Queue (deque) Enqueue: 0.001012 s
Queue (deque) Peek: 0.000000 s
Queue (deque) Dequeue: 0.001754 s
Priority Queue (heapq) Push: 0.003000 s
Priority Queue (heapq) Peek: 0.000000 s
Priority Queue (heapq) Pop: 0.023019 s
```

For all three of stacks, queues and priority queues, the peek method is almost instantaneous (as is logical for a O(1) method). The push/enqueue methods consistently vary in time by about 0.0002 seconds between stacks and queues, however the priority queue push takes much longer than these two due to the need to find nodes that are smaller than the given values. The pop method takes comparable amounts of time for stacks and queues, but this takes around 10x as long for priority queues as any child nodes need to be reassigned.

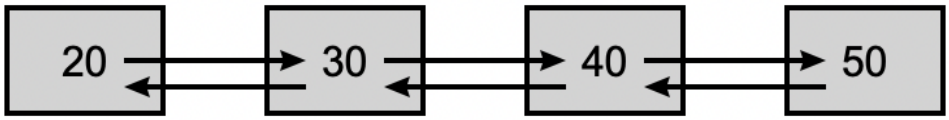
Task 2: Test results (double linked)



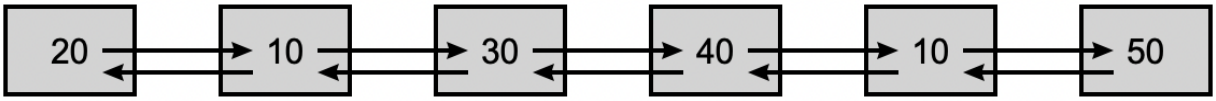
Insertion at index 0



Search for 30

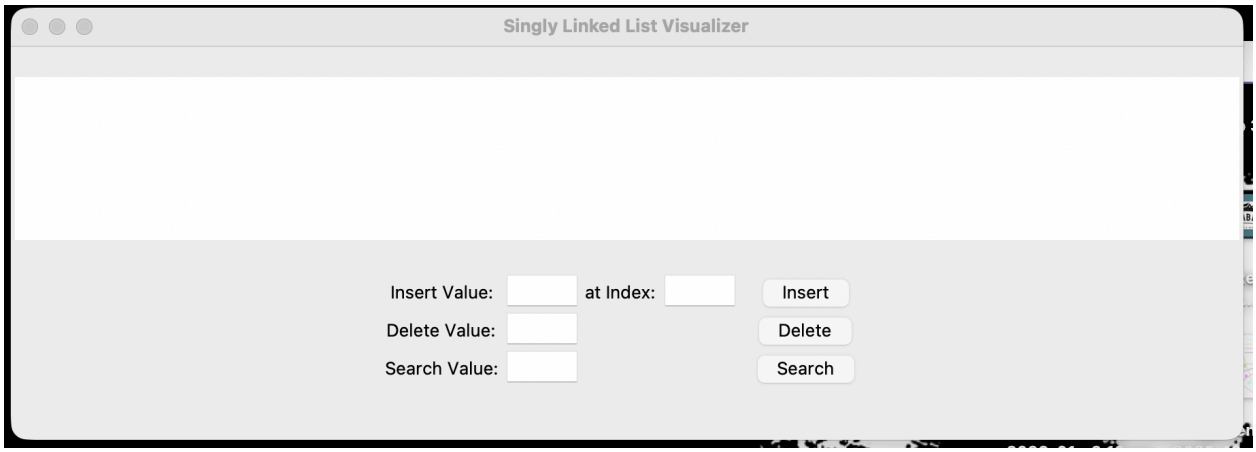


10 deleted

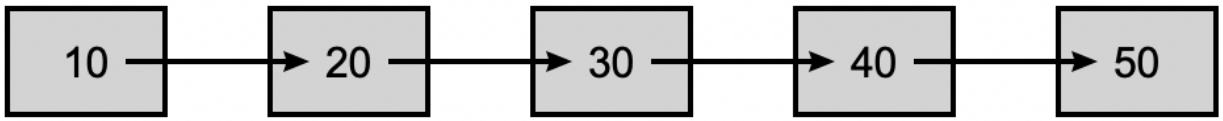


10 deleted once

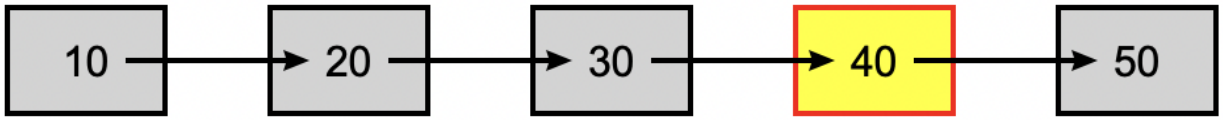
Test results (single linked):



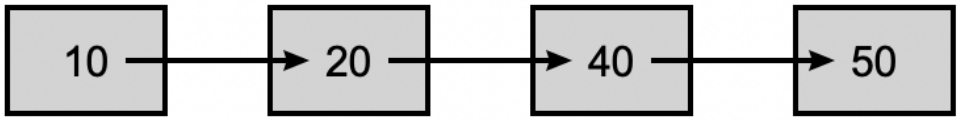
GUI layout



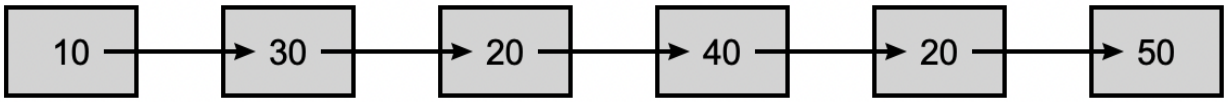
Insert at index 0



Search for 40



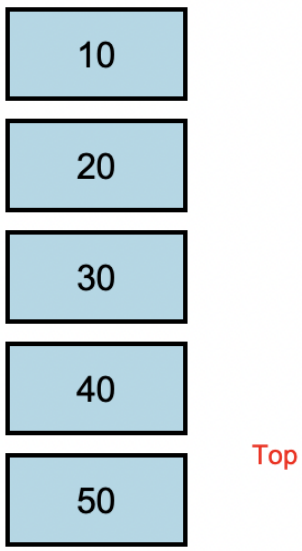
30 deleted



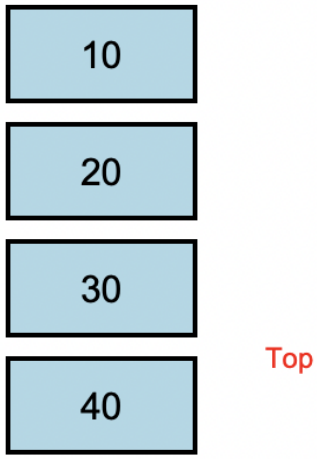
30 deleted once

Task 3: Test results (stack)

Pushed values

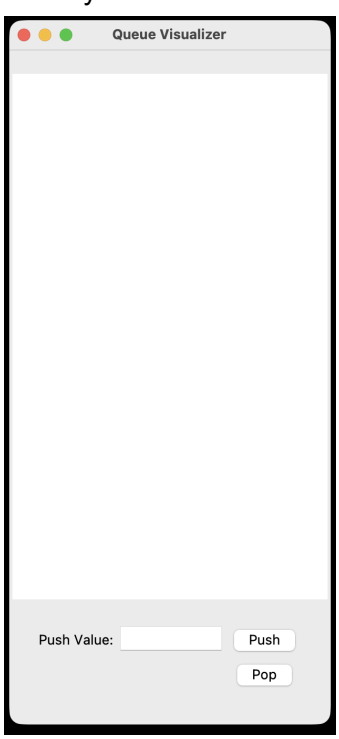


Popped value

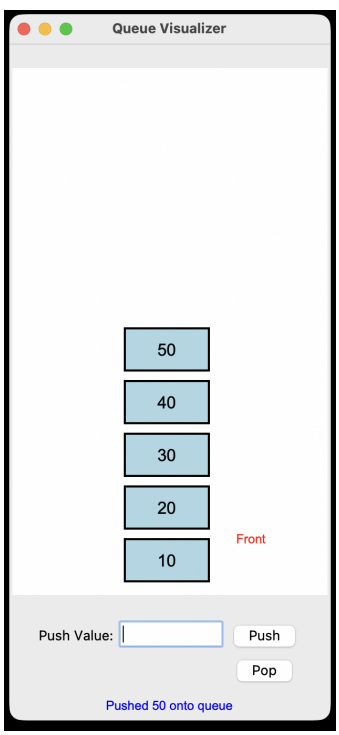


Test results (queue)

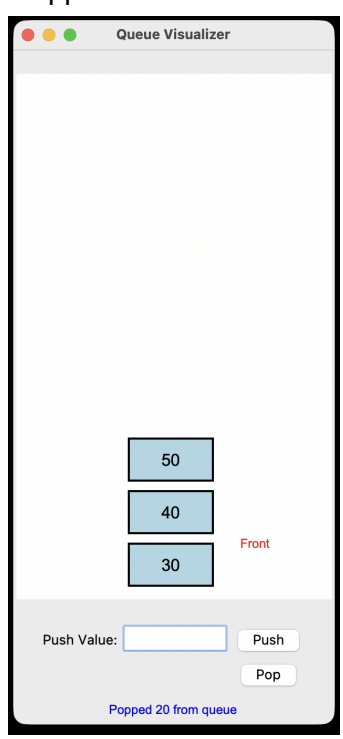
GUI layout



Pushed values

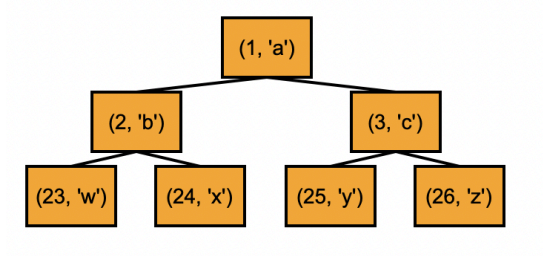


Popped values

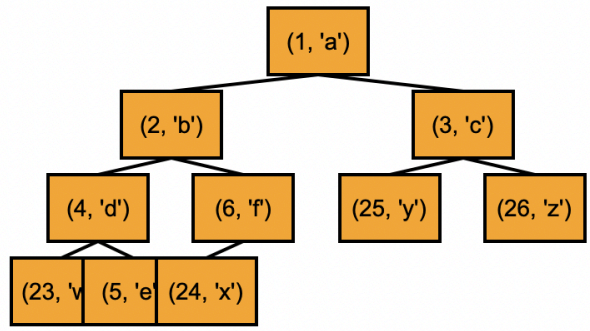
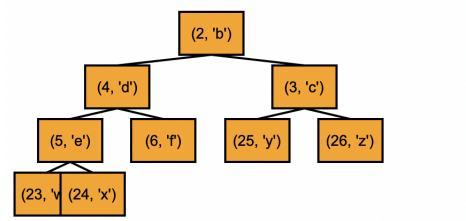
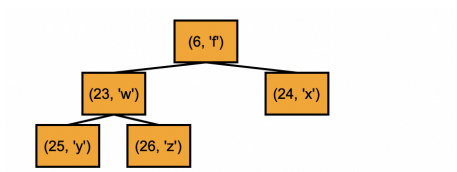
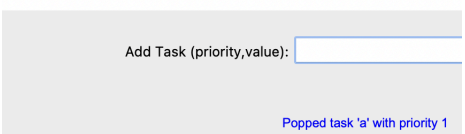
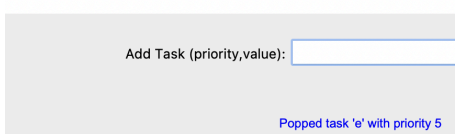
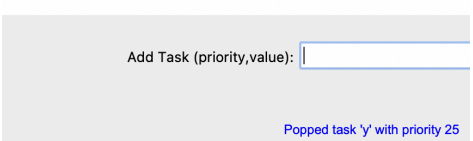


Task 4: Test results

Add tasks



Add tasks (in between)



Pop tasks